ADR 0002: Evaluation sandboxes through the internal OpenShell gateway

- **Status:** Proposed
- **Date:** 2026-10-19
- **Deciders:** EvalHub, TrustyAI Service Operator, OpenShift AI platform and Security
- **Strategy:** RHAISTRAT-2289
- **Depends on:** shared internal OpenShell gateway proposal, PR 174, reviewed at `57bc18bca3c487bc56b3be18ca147f112141d37b`
- **Proposed partial supersession:** ADR 0001 in EvalHub PR 1059, itself still Proposed
- **Implementation plan:** OpenShell evaluation plan

Context

Agentic evaluation frameworks execute untrusted code, create child environments and require remote execution and observation. EvalHub must contain their access to tenants, credentials, cluster APIs, network destinations and compute capacity, while preserving ordinary evaluations.

The earlier EvalHub proposal uses an ephemeral namespace and a Kubernetes API broker sidecar as its initial boundary, with OpenShell added later. Platform PR 174 instead proposes a shared internal OpenShell gateway and makes it the baseline for sandbox-required EvalHub execution. Implementing both independent compute control planes would duplicate security, lifecycle and operational work.

The shared gateway does not remove EvalHub's multi-tenancy obligations. Customer-supplied code remains untrusted even when executing as a bounded service operation. A common service identity across all evaluation workspaces, or a credential in the untrusted process, would undermine per-evaluation containment.

The pinned OpenShell Kubernetes driver supports managed workspace namespaces, separate workload/supervisor Pods and PVC-backed workspace storage. The authorization implementation, API schema and PR 174 review show that workload identity, complete policy admission, namespace preparation and compatibility require explicit integration. Workspace templates alone do not pin the policy/provider combination.

The strategy's 2026-10-13 refinement adds resource-kind ceilings, Helm provisioning and synthetic canary needs. These cannot be assumed equivalent to simple OpenShell child creation.

Decision

1. Use the shared gateway for mandatory sandbox execution

Use one platform-managed internal gateway per OpenShift AI installation, as proposed by PR 174. Sandboxing is resolved from trusted provider/task configuration and installation capabilities. Sandbox-required evaluation benchmarks run their evaluator and children as OpenShell sandboxes, never as an unsandboxed Job fallback. The existing Job backend remains for ordinary benchmarks, including those in mixed collections; the evaluation aggregates status and results across both backends.

`standard` and `kata` are approved OpenShell runtime profiles. A provider requiring Kata fails closed if the profile is unavailable or unqualified. Local process execution also rejects mandatory-sandbox providers. There is no request-time escape hatch to lower a provider's requirements.

2. Preserve a versioned evaluation lifecycle, delegate compute lifecycle

Retain `SandboxedEvaluation` as the asynchronous cross-repository contract. EvalHub persists the accepted request and dispatch intent; TrustyAI reconciles it, reports progress and owns finalizer-driven cancellation and cleanup. A platform-owned provisioning service creates workspace membership, approved templates and credential bindings through the gateway.

Allocate one ephemeral workspace and managed namespace per evaluation attempt, with one primary sandbox per sandbox-required benchmark and bounded child sandboxes. Record immutable tenant, evaluation, workspace and namespace identities. OpenShell owns managed namespace creation and deletion; TrustyAI does not create an independent namespace or primary Kubernetes Job for this path.

The platform provisioning service, gateway, admission components and Agent Sandbox controller have named operational ownership and versioned readiness. The provisioning service holds the necessary OpenShell administrative authority; EvalHub and its brokers do not receive it.

3. Keep trusted adapters outside evaluated code

Deploy `eval-sandbox-broker` as a trusted per-attempt service in a control namespace. It authenticates untrusted requests using an attempt-bound capability and uses a unique workspace-user identity for that attempt. It cannot operate in another evaluation workspace. No gateway credential or production Kubernetes pod-creation token is passed to the agent.

The broker reconstructs approved OpenShell requests from supported Kubernetes-shaped operations. It is not a production Kubernetes API reverse proxy. Support only provider-qualified semantics, including any required streaming channel behavior; reject unsupported Pod fields, kinds and namespace references explicitly. Server admission independently enforces the resolved image, template, policy, provider bindings, runtime, resource and lifecycle constraints.

For Harbor, prove an OpenShell-native framework plugin or bounded virtual compatibility implementation against the actual Helm lifecycle. Do not claim arbitrary Helm, Services, Secrets, controllers or Kubernetes networking are supported by a Pod-to-sandbox mapping. A failed compatibility proof blocks that provider until the requirements or implementation change through review.

4. Enforce namespace and policy readiness before execution

Use managed workspace mode from PR 174, with a unique immutable gateway ID and protected namespace ownership mapping. Reject conflicting pre-created namespaces. Add a platform/upstream preparation barrier so no workload or supervisor executes before SCC allocation, approved PSA/SCC, quota, limits, storage and effective ingress/egress policy are ready.

The selected OpenShell release must support that barrier, or a fail-closed admission implementation must prevent premature execution and support idempotent recovery. Applying quota after first sandbox startup is insufficient. An operator-workspace-mode alternative requires an explicit amendment to the platform decision.

Independently restrict the managed-mode gateway and Agent Sandbox controller's cluster authority to authorized namespace/resource operations using admission or an equivalent enforceable boundary. Labels alone do not constrain a compromised cluster-wide credential. Protect readiness and ownership metadata from consumers and evaluated workloads.

Pin the complete effective execution profile on the server. A template reference alone is insufficient: validate inline settings, policy, provider attachments, runtime selection, restarts and mutations. Require enforced kernel policy, manual policy expansion approval and denial on interceptor unavailability. Disable exposed-service routing unless an authenticated cross-tenant-safe implementation is qualified.

Constrain both workload and supervisor network paths. Deny API-server and metadata access and unapproved endpoints. Resolve the driver's permissive supervisor egress using a supported policy change or higher-precedence cluster control; ordinary additive NetworkPolicy cannot remove an existing allowance. Verify effective traffic behavior on the supported CNI.

5. Preserve tenant and credential boundaries

Maintain the original authorized tenant and MLflow workspace independently of the ephemeral compute namespace. Preserve EH-0002 and the credential-hiding objective of EH-0004.

For the sandbox backend, use approved workspace-scoped OpenShell credential delivery and a trusted remote integration relay for EvalHub callbacks, MLflow/OCI and behavior requiring EvalHub-specific adaptation. The relay binds every call to the tenant, evaluation, benchmark and attempt and reconstructs trusted upstream requests. Ordinary Jobs retain their sidecar.

Read only explicitly delegated tenant secret references. Do not mount original secrets or asynchronous user bearer tokens into evaluated code. Synthetic canary injection is a separate bounded input capability for approved benchmark profiles. Negative-control test targets remain inside the outer sandbox, with no real Kubernetes authentication token or weakening of outer containment.

6. Enforce aggregate capacity and bounded lifetime

Intersect provider-declared resource kinds with an administrator ceiling and implemented operations. Enforce per-sandbox and aggregate evaluation/tenant/installation limits, including supervisors, init peaks, brokers, storage, streams, fan-out and depth.

On Kueue-enabled installations, obtain queue admission for the declared footprint before launching sandboxes and handle eviction explicitly. Existing Job admission does not automatically apply to OpenShell. Namespace quotas remain necessary but cannot substitute for queue fairness. A compatible admission adapter is a delivery prerequisite.

Permit only automatically provisioned, short-lived workspace PVCs under a qualified storage deletion contract. This changes the prior `emptyDir`-only scope. Persistent user storage, retained workspaces, GPU passthrough and arbitrary runtime selection remain out of scope.

7. Make cleanup and provenance part of success

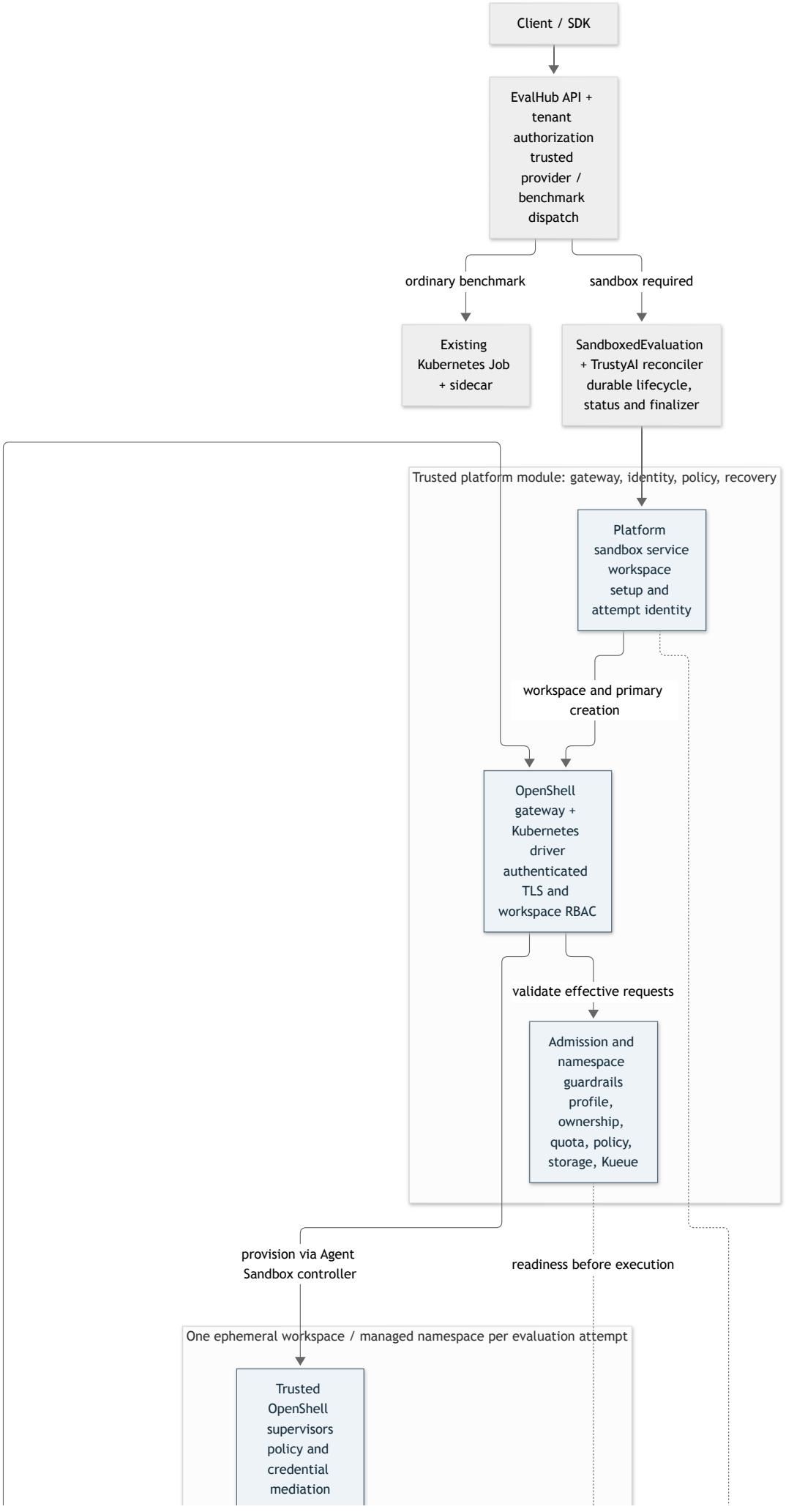
Persist lifecycle intent and external identifiers before progressing. Freeze child creation on terminal events; terminate primary/children, close streams, export available results, remove workspace-scoped

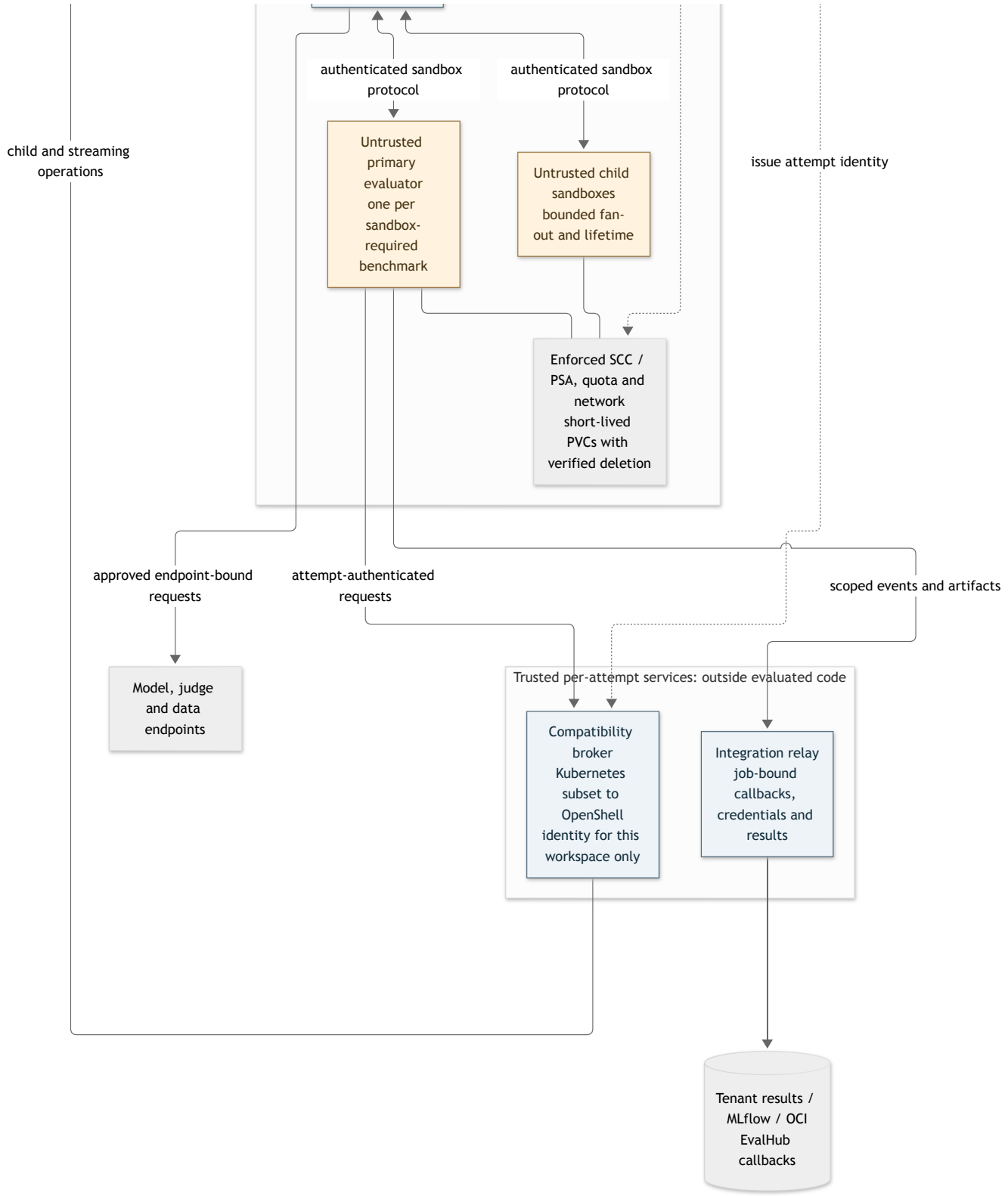
objects, revoke memberships/identity and delete the workspace. OpenShell removes the namespace. Verify namespace and storage cleanup; stop/suspend is not deletion.

A platform sweeper handles TTL expiry and abandoned resources. Finalizers retry with bounded backoff. A stalled cleanup raises an explicit condition and durable orphan record; a timeout never becomes a successful cleanup assertion. Cross-namespace garbage collection is not an ownership mechanism.

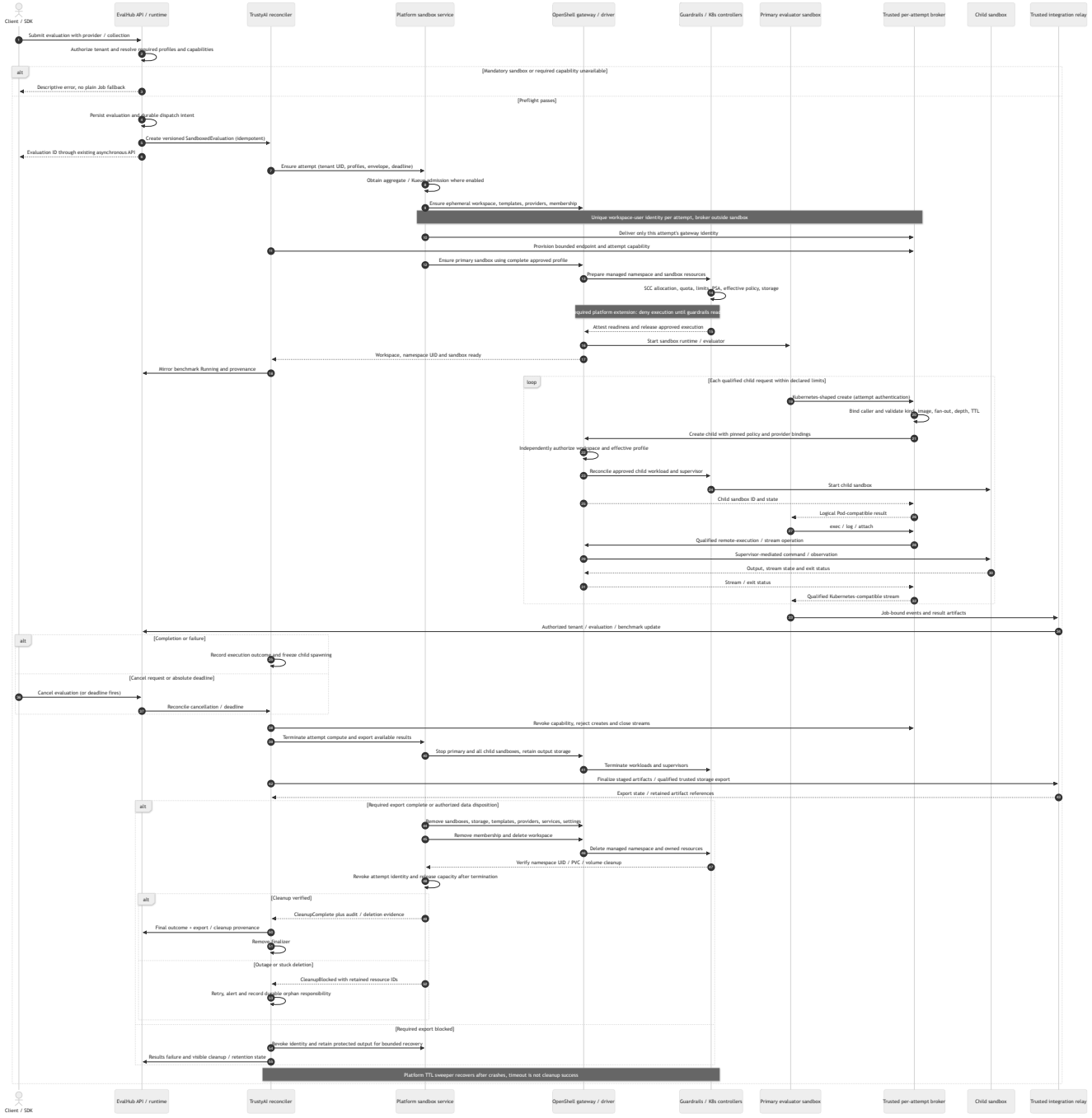
Record gateway and component versions, tenant/attempt IDs, workspace/namespace UID, primary/child IDs, template/image digests, effective policy and runtime, resource limits, audit references and cleanup state in evaluation provenance. Export required artifacts before normal namespace removal.

Architecture and execution


Client / SDK
EvalHub API + tenant authorization trusted provider / benchmark dispatch
ordinary benchmark
sandbox required
Existing Kubernetes Job + sidecar
SandboxedEvaluation + TrustyAI reconciler durable lifecycle, status and finalizer
Trusted platform module: gateway, identity, policy, recovery
Platform sandbox service workspace setup and attempt identity
workspace and primary creation
OpenShell gateway + Kubernetes driver authenticated TLS and workspace RBAC
validate effective requests
Admission and namespace guardrails profile, ownership, quota, policy, storage, Kueue
provision via Agent Sandbox controller
readiness before execution
One ephemeral workspace / managed namespace per evaluation attempt
Trusted OpenShell supervisors policy and credential mediation



Architecture source.



Sequence source. The sequence covers one sandbox-required benchmark; additional benchmarks share the evaluation attempt's bounded workspace and lifecycle.

Alternatives considered

- **Independent namespace/Kubernetes broker with optional OpenShell:** preserves Kubernetes semantics more directly but duplicates compute, policy and lifecycle ownership. Rejected as the default under PR 174. A separately reviewed provider-specific exception is a different architecture, never automatic fallback.
- **One workspace and broker identity for all EvalHub jobs:** reduces churn but combines tenants and gives a compromised broker access to other evaluations. Rejected.
- **Untrusted evaluator as a plain Job, sandbox only its children:** leaves the evaluator itself outside the required boundary and does not cover adversarial framework code. Rejected for mandatory-sandbox providers.

- **Broker sidecar inside an OpenShell sandbox:** assumes a multi-container credential/trust model not supplied by this integration and places privileged mediation beside untrusted code. Rejected in favor of an external per-attempt adapter.
- **Operator workspace mode:** makes pre-provisioning controls and namespace-scoped RBAC easier, but changes PR 174's managed-mode ownership. Retain as a reviewed alternative if the required managed-mode barrier cannot be delivered.
- **Per-consumer/customer-managed gateway:** appropriate where a distinct administrative or compromise boundary is required. Not the supported shared internal path selected here.

Consequences

The design gives EvalHub a common platform sandbox runtime and removes its production Kubernetes API proxy and independent compute lifecycle. Per-evaluation workspaces, identities and finalizers preserve containment and recovery while enabling platform-wide policy and audit.

It makes the shared gateway, privileged platform provisioning service, interceptor, identity issuer, Agent Sandbox controller, storage and admission integration mandatory dependencies. A compromised shared control plane can affect multiple internal consumers. Workspace boundaries do not eliminate that risk. Standard containers still share the host kernel; Kata reduces a workload risk without isolating the whole control plane.

Compatibility translation, workspace churn, two-Pod sandbox footprint, storage cleanup and Kueue integration can dominate complexity and latency. The project must measure supported concurrency and prove representative frameworks before committing provider coverage or delivery dates.

On acceptance, update the strategy and partially supersede ADR 0001's compute ownership, broker placement, optional OpenShell phasing, local warn-and-allow, Job scheduling and storage decisions. Keep its dedicated-evaluation isolation, governed provider selection, image controls, resource limits and lifecycle intent. Both records remain Proposed until the responsible reviewers accept them.

Acceptance gates

Approval of this design does not assert implementation readiness. Enable a provider only after its compatibility matrix, namespace readiness and cluster-authority controls, per-attempt identity, policy/credential tests, queue admission, storage reclamation, restart/teardown tests and performance envelope pass. The implementation plan defines phase owners, proposed timing targets and detailed traceability.